Fjalorët dhe Set në Python

Alion Emini

kurseinformatike.com

Përmbajtja

1. Fjalorët (Dictionaries)
2. Set
3. Pickle: Serializimi i objekteve
4. JSON në Python
5. Përmbledhje

1 Fjalorët (Dictionaries)
- Krijimi dhe aksesimi
- Shtimi, ndryshimi, fshirja
- Tipe të përziera dhe fjalor i folezuar
- Cikli `for` dhe metodat kryesore
- Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.
  - Përkufizimi = vlera.

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.
  - Përkufizimi = vlera.
- Në aplikacione:

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.
  - Përkufizimi = vlera.
- Në aplikacione:
  - ID → emër punonjësi

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.
  - Përkufizimi = vlera.
- Në aplikacione:
  - ID → emër punonjësi
  - emër → nr. telefoni

Çfarë është një fjalor në Python?

Përkufizim

Një **fjalor** ruan çifte **çelës–vlerë**.

- Çelësi (*key*) identifikon unikisht hyrjen.
- Vlera (*value*) është e dhëna që duam të ruajmë.
- Kërkojmë me çelësin, jo me indeks.

- Analogia: fjalori i gjuhës.
  - Fjala = çelësi.
  - Përkufizimi = vlera.
- Në aplikacione:
  - ID → emër punonjësi
  - emër → nr. telefoni
  - username → profil përdoruesi

1 Fjalorët (Dictionaries)
Krijimi dhe aksesimi
Shtimi, ndryshimi, fshirja
Tipe të përziera dhe fjalor i folezuar
Cikli `for` dhe metodat kryesore
Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Krijimi i një fjalori

```
phonebook = {
    'Chris':  '555-1111',
    'Katie':  '555-2222',
    'Joanne': '555-3333'
}
```

- Kllapa kaçurrelë { }.

Krijimi i një fjalori

```
phonebook = {
    'Chris':  '555-1111',
    'Katie':  '555-2222',
    'Joanne': '555-3333'
}
```

- Kllapa kaçurrelë { }.
- Çdo hyrje: `key:  value` e ndarë me presje.

Krijimi i një fjalori

```
phonebook = {
    'Chris':  '555-1111',
    'Katie':  '555-2222',
    'Joanne': '555-3333'
}
```

- Kllapa kaçurrelë `{ }`.
- Çdo hyrje: `key:  value` e ndarë me presje.
- Çelësat zakonisht `str`, por mund të jenë edhe `int`, `tuple`, ... (por **immutable**).

Krijimi i një fjalori

```
phonebook = {
    'Chris':  '555-1111',
    'Katie':  '555-2222',
    'Joanne': '555-3333'
}
```

- Kllapa kaçurrelë `{ }`.
- Çdo hyrje: `key:  value` e ndarë me presje.
- Çelësat zakonisht `str`, por mund të jenë edhe `int`, `tuple`, ... (por **immutable**).

Krijimi i një fjalori

```
phonebook = {
    'Chris':  '555-1111',
    'Katie':  '555-2222',
    'Joanne': '555-3333'
}
```

- Kllapa kaçurrelë `{ }`.
- Çdo hyrje: `key:  value` e ndarë me presje.
- Çelësat zakonisht `str`, por mund të jenë edhe `int`, `tuple`, ... (por **immutable**).

Kujdes

- Të gjithë çelësat duhet të jenë **hashable** / **immutable**.
- `list` nuk mund të jetë çelës.

Marrja e një vlere

```
1 print(phonebook['Chris'])
2 print(phonebook['Katie'])
```

Marrja e një vlere

```
print(phonebook['Chris'])
print(phonebook['Katie'])
```

```
555-1111
555-2222
```

- Nëse çelësi ekziston ⇒ kthen vlerën.

Marrja e një vlere

```python
print(phonebook['Chris'])
print(phonebook['Katie'])
```

```
555-1111
555-2222
```

- Nëse çelësi ekziston ⇒ kthen vlerën.
- Nëse çelësi mungon ⇒ `KeyError`.

Marrja e një vlere

```
print(phonebook['Chris'])
print(phonebook['Katie'])
```

```
555-1111
555-2222
```

- Nëse çelësi ekziston ⇒ kthen vlerën.
- Nëse çelësi mungon ⇒ `KeyError`.
- Krahasimi i `str` është **case-sensitive**: `'Chris'` ≠ `'chris'`.

in dhe not in

```
if 'Chris' in phonebook:
    print(phonebook['Chris'])

if 'Joanne' not in phonebook:
    print('Joanne is not found')
```

in dhe not in

```
if 'Chris' in phonebook:
    print(phonebook['Chris'])

if 'Joanne' not in phonebook:
    print('Joanne is not found')
```

- Përdor `in` për të shmangur `KeyError`.

in dhe not in

```
if 'Chris' in phonebook:
    print(phonebook['Chris'])

if 'Joanne' not in phonebook:
    print('Joanne is not found')
```

- Përdor `in` për të shmangur `KeyError`.
- Shumë i dobishëm para fshirjes, leximit, etj.

1 Fjalorët (Dictionaries)
- Krijimi dhe aksesimi
- Shtimi, ndryshimi, fshirja
- Tipe të përziera dhe fjalor i folezuar
- Cikli `for` dhe metodat kryesore
- Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Shtimi dhe përditësimi

```
phonebook['Joe']   = '555-0123'   # shtim
phonebook['Chris'] = '555-4444'   # ndryshim vlere
```

- Nëse çelësi nuk ekziston ⇒ shtohet hyrje e re.

Shtimi dhe përditësimi

```
phonebook['Joe']   = '555-0123'   # shtim
phonebook['Chris'] = '555-4444'   # ndryshim vlere
```

- Nëse çelësi nuk ekziston ⇒ shtohet hyrje e re.
- Nëse ekziston ⇒ vlera e vjetër mbishkruhet.

Shtimi dhe përditësimi

```
phonebook['Joe']   = '555-0123'   # shtim
phonebook['Chris'] = '555-4444'   # ndryshim vlere
```

- Nëse çelësi nuk ekziston ⇒ shtohet hyrje e re.
- Nëse ekziston ⇒ vlera e vjetër mbishkruhet.
- Fjalorët janë **mutable**.

Fshirja me del

```
del phonebook['Chris']
```

Fshirja me del

```
del phonebook['Chris']
```

- Heq si çelësin, ashtu edhe vlerën.

Fshirja me del

```
del phonebook['Chris']
```

- Heq si çelësin, ashtu edhe vlerën.
- Nëse çelësi mungon ⇒ `KeyError`.

Fshirja me del

```
del phonebook['Chris']
```

- Heq si çelësin, ashtu edhe vlerën.
- Nëse çelësi mungon ⇒ `KeyError`.

Fshirja me del

```
del phonebook['Chris']
```

- Heq si çelësin, ashtu edhe vlerën.
- Nëse çelësi mungon ⇒ KeyError.

```
if 'Chris' in phonebook:
    del phonebook['Chris']
```

- Praktikë e sigurt: kontrollo me in ose përdor pop me default.

Numri i elementëve: len()

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
print(len(phonebook))     # 2
```

- Kthen numrin e çifteve *çelës–vlerë*.

Numri i elementëve: len()

```
phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}
print(len(phonebook))     # 2
```

- Kthen numrin e çifteve *çelës–vlerë*.
- Nuk varet nga madhësia e vlerave.

1 Fjalorët (Dictionaries)
- Krijimi dhe aksesimi
- Shtimi, ndryshimi, fshirja
- Tipe të përziera dhe fjalor i folezuar
- Cikli `for` dhe metodat kryesore
- Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Vlera komplekse (lista notash)

```
test_scores = {
    'Kayla':  [88, 92, 100],
    'Luis':   [95, 74, 81],
    'Sophie': [72, 88, 91]
}

print(test_scores['Sophie'])  # [72, 88, 91]
```

- Çelësat: `str`.

Vlera komplekse (lista notash)

```python
test_scores = {
    'Kayla':  [88, 92, 100],
    'Luis':   [95, 74, 81],
    'Sophie': [72, 88, 91]
}

print(test_scores['Sophie'])  # [72, 88, 91]
```

- Çelësat: `str`.
- Vlerat: lista `int`-ash.

Vlera komplekse (lista notash)

```
test_scores = {
    'Kayla':  [88, 92, 100],
    'Luis':   [95, 74, 81],
    'Sophie': [72, 88, 91]
}

print(test_scores['Sophie'])  # [72, 88, 91]
```

- Çelësat: `str`.
- Vlerat: lista `int`-ash.
- E natyrshme: emri → të gjitha notat.

Tipe të përziera

```
mixed_up = {
    'abc': 1,
    999: 'yada yada',
    (3, 6, 9): [3, 6, 9]
}
```

- Çelës `str`, `int`, `tuple`.

Tipe të përziera

```
mixed_up = {
    'abc': 1,
    999: 'yada yada',
    (3, 6, 9): [3, 6, 9]
}
```

- Çelës `str`, `int`, `tuple`.
- Vlera `int`, `str`, `list`.

Tipe të përziera

```
mixed_up = {
    'abc': 1,
    999: 'yada yada',
    (3, 6, 9): [3, 6, 9]
}
```

- Çelës `str`, `int`, `tuple`.
- Vlera `int`, `str`, `list`.
- Në praktikë: zgjidh stil të qëndrueshëm për lexueshmëri.

Fjalor si record (punonjës)

```
employee = {
    'name':     'Kevin Smith',
    'id':       12345,
    'payrate': 25.75
}
```

- `employee['name']` është vetëshpjegues.

Fjalor si record (punonjës)

```
employee = {
    'name':     'Kevin Smith',
    'id':       12345,
    'payrate':  25.75
}
```

- `employee['name']` është vetëshpjegues.
- Mund të shtojmë fusha të reja pa prishur kodin tjetër.

Fjalor si record (punonjës)

```
employee = {
    'name':     'Kevin Smith',
    'id':       12345,
    'payrate':  25.75
}
```

- `employee['name']` është vetëshpjegues.
- Mund të shtojmë fusha të reja pa prishur kodin tjetër.
- Struktura tipike: listë me fjalorë punonjësish.

Fjalorë të folezuar

```python
students = {
  'A101': {'name': 'Arta',  'scores': [90, 85, 92], 'active': True},
  'B202': {'name': 'Bledi', 'scores': [78, 92],     'active': False},
}

print(students['A101']['name'])        # Arta
print(students['A101']['scores'][0])   # 90
```

- Modelim hierarkik: ID → info i detajuar.

Fjalorë të folezuar

```python
students = {
  'A101': {'name': 'Arta',  'scores': [90, 85, 92], 'active': True},
  'B202': {'name': 'Bledi', 'scores': [78, 92],     'active': False},
}

print(students['A101']['name'])        # Arta
print(students['A101']['scores'][0])   # 90
```

- Modelim hierarkik: ID → info i detajuar.
- Shumë i përshtatshëm për të dhëna nga JSON/APIs.

1 Fjalorët (Dictionaries)
- Krijimi dhe aksesimi
- Shtimi, ndryshimi, fshirja
- Tipe të përziera dhe fjalor i folezuar
- Cikli `for` dhe metodat kryesore
- Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Iterim me for

```
for key in phonebook:
    print(key, phonebook[key])
```

- Iterimi default: vetëm çelësat.

Iterim me for

```
for key in phonebook:
    print(key, phonebook[key])
```

- Iterimi default: vetëm çelësat.
- Përdor çelësin për të marrë vlerën.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.
- `items()` çifte `(key, value)`.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.
- `items()` çifte `(key, value)`.
- `keys()` vetëm çelësat.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.
- `items()` çifte `(key, value)`.
- `keys()` vetëm çelësat.
- `values()` vetëm vlerat.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.
- `items()` çifte `(key, value)`.
- `keys()` vetëm çelësat.
- `values()` vetëm vlerat.
- `pop(key, default)` kthen *dhe* heq.

Përmbledhje metodash kryesore

- `clear()` zbraz fjalorin.
- `get(key, default)` akses i sigurt pa `KeyError`.
- `items()` çifte `(key, value)`.
- `keys()` vetëm çelësat.
- `values()` vetëm vlerat.
- `pop(key, default)` kthen *dhe* heq.
- `popitem()` heq çiftin e fundit (LIFO).

get: akses i sigurt

```
value = phonebook.get('Katie', 'Entry not found')
print(value)    # 555-2222

value = phonebook.get('Andy', 'Entry not found')
print(value)    # Entry not found
```

- Ideale kur çelësi mund të mungojë.

get: akses i sigurt

```
value = phonebook.get('Katie', 'Entry not found')
print(value)    # 555-2222

value = phonebook.get('Andy', 'Entry not found')
print(value)    # Entry not found
```

- Ideale kur çelësi mund të mungojë.
- Shmang `try/except` për raste të thjeshta.

items, keys, values

```
for k, v in phonebook.items():
    print(k, v)

for k in phonebook.keys():
    print(k)

for v in phonebook.values():
    print(v)
```

- Kthejnë **dictionary views** (dinamikë, jo lista kopje).

items, keys, values

```
for k, v in phonebook.items():
    print(k, v)

for k in phonebook.keys():
    print(k)

for v in phonebook.values():
    print(v)
```

- Kthejnë **dictionary views** (dinamikë, jo lista kopje).
- Praktikë standarde për iterim të pastër mbi fjalorë.

1 Fjalorët (Dictionaries)
- Krijimi dhe aksesimi
- Shtimi, ndryshimi, fshirja
- Tipe të përziera dhe fjalor i folezuar
- Cikli `for` dhe metodat kryesore
- Dictionary comprehensions

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Dictionary comprehension: bazë

```
numbers = [1, 2, 3, 4]
squares = {n: n**2 for n in numbers}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16}
```

- Sintaksë kompakte për të ndërtuar fjalorë.

Dictionary comprehension: bazë

```
numbers = [1, 2, 3, 4]
squares = {n: n**2 for n in numbers}
print(squares)  # {1: 1, 2: 4, 3: 9, 4: 16}
```

- Sintaksë kompakte për të ndërtuar fjalorë.
- Forma: `{key_expr: value_expr for ...}`.

Me if filtrues

```
populations = {
  'NY': 8398748,
  'LA': 3990456,
  'Chicago': 2705994,
  'Houston': 2325502,
  'Phoenix': 1660272
}

largest = {k: v for k, v in populations.items() if v > 2
    _000_000}
```

- Ndërto *në një hap* dhe filtro elementët.

1 Fjalorët (Dictionaries)

2 Set
- Krijimi, shtimi, heqja
- Operacione me sete
- Set comprehensions

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.
- Rend i pakarantuar.

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.
- Rend i pakarantuar.
- Ideal për:

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.
- Rend i pakarantuar.
- Ideal për:
  - heqje duplikimesh,

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.
- Rend i pakarantuar.
- Ideal për:
  - heqje duplikimesh,
  - operacione bashkimi/prerje,

Çfarë është një *set*?

Përkufizim

Një **set** është koleksion **vlerash unike**, i parenditur, i ngjashëm me setet matematikore.

- Pa duplikime.
- Rend i pakarantuar.
- Ideal për:
  - heqje duplikimesh,
  - operacione bashkimi/prerje,
  - kërkime shumë të shpejta `in`.

1 Fjalorët (Dictionaries)

2 Set
- Krijimi, shtimi, heqja
- Operacione me sete
- Set comprehensions

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Krijimi i një *set*-i

```
s = set()                    # set bosh
s = set([1, 2, 3])           # nga listë
s = set('abc')               # {'a', 'b', 'c'}
s = set(['one','two'])       # elemente string
```

- Duplikimet hiqen automatikisht.

Krijimi i një *set*-i

```
s = set()                    # set bosh
s = set([1, 2, 3])           # nga listë
s = set('abc')               # {'a', 'b', 'c'}
s = set(['one','two'])       # elemente string
```

- Duplikimet hiqen automatikisht.
- Elementët duhet të jenë **hashable** (si çelësat e fjalorit).

Shtimi dhe heqja

```
s = set()
s.add(1)
s.add(2)
s.update([2, 3, 4])    # shton disa

s.remove(1)            # KeyError nëse mungon
s.discard(99)          # pa gabim nëse mungon
s.clear()              # zbraz set-in
```

- add, update për shtim.

Shtimi dhe heqja

```
s = set()
s.add(1)
s.add(2)
s.update([2, 3, 4])    # shton disa

s.remove(1)            # KeyError nëse mungon
s.discard(99)          # pa gabim nëse mungon
s.clear()              # zbraz set-in
```

- `add`, `update` për shtim.
- `remove` vs `discard`: sjellje ndryshe në mungesë.

Iterim dhe in

```
s = set(['a', 'b', 'c'])
for x in s:
    print(x)

if 'a' in s:
    print('ka a')
```

- Rend i pakarantuar.

Iterim dhe in

```
s = set(['a', 'b', 'c'])
for x in s:
    print(x)

if 'a' in s:
    print('ka a')
```

- Rend i pakarantuar.
- in është shumë efikas për kërkim.

1 Fjalorët (Dictionaries)

2 Set
- Krijimi, shtimi, heqja
- Operacione me sete
- Set comprehensions

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Union (*bashkimi*)

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a.union(b))   # {1,2,3,4,5,6}
print(a | b)        # e njëjta gjë
```

Intersection (*prerja*)

```
print(a.intersection(b))   # {3, 4}
print(a & b)               # e njëjta
```

Difference (*diferenca*)

```
print(a.difference(b))   # {1, 2}
print(a - b)             # e njëjta
```

Symmetric difference

```
print(a.symmetric_difference(b))   # {1,2,5,6}
print(a ^ b)                       # e njëjta
```

Nënsete dhe mbisete

```
set1 = {1, 2, 3, 4}
set2 = {2, 3}

print(set2.issubset(set1))      # True
print(set1.issuperset(set2))    # True
print(set2 <= set1, set1 >= set2)
```

1 Fjalorët (Dictionaries)

2 Set
- Krijimi, shtimi, heqja
- Operacione me sete
- Set comprehensions

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Set comprehensions

```
nums = [1, 2, 2, 3, 4, 5]
squares = {n**2 for n in nums if n % 2 == 1}
print(squares)  # {1, 9, 25}
```

- Sintaksë: `{expr for item in iterable if cond}`.

Set comprehensions

```
nums = [1, 2, 2, 3, 4, 5]
squares = {n**2 for n in nums if n % 2 == 1}
print(squares)  # {1, 9, 25}
```

- Sintaksë: `{expr for item in iterable if cond}`.
- Duplikimet hiqen automatikisht.

1 Fjalorët (Dictionaries)

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Çfarë është pickling?

Përkufizim

Pickling: objekt Python → rrjedhë bajtësh (për skedar).
Unpickling: bajtët → objekt Python.

- Ruajtje e strukturave komplekse (`dict`, `list`, `set`, etj.).

Çfarë është pickling?

Përkufizim

Pickling: objekt Python → rrjedhë bajtësh (për skedar).
Unpickling: bajtët → objekt Python.

- Ruajtje e strukturave komplekse (`dict`, `list`, `set`, etj.).
- E dobishme për cache lokale, ruajtje të gjendjes së programit.

Shembull: pickling

```
import pickle

phonebook = {'Chris':'555-1111', 'Katie':'555-2222'}

with open('phonebook.dat', 'wb') as f:
    pickle.dump(phonebook, f, protocol=pickle.HIGHEST_PROTOCOL)
```

Shembull: unpickling

```
import pickle

with open('phonebook.dat', 'rb') as f:
    pb = pickle.load(f)

print(pb)
```

Kujdes sigurie

Mos harro

- `pickle.load` **mund të ekzekutojë kod**.
- Mos bëj unpickling të të dhënave nga burime të pa-besueshme.
- Për shkëmbim publik: prefero **JSON**.

1 Fjalorët (Dictionaries)

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.
- Format tekstual, i lehtë, i lexueshëm nga njerëzit.

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.
- Format tekstual, i lehtë, i lexueshëm nga njerëzit.
- I pavarur nga gjuha: përdoret nga Python, JavaScript, Java, etj.

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.
- Format tekstual, i lehtë, i lexueshëm nga njerëzit.
- I pavarur nga gjuha: përdoret nga Python, JavaScript, Java, etj.
- Bazohet në:

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.
- Format tekstual, i lehtë, i lexueshëm nga njerëzit.
- I pavarur nga gjuha: përdoret nga Python, JavaScript, Java, etj.
- Bazohet në:
  - objekte: `{ "key": value, ... }`

Çfarë është JSON?

- **JSON** = JavaScript Object Notation.
- Format tekstual, i lehtë, i lexueshëm nga njerëzit.
- I pavarur nga gjuha: përdoret nga Python, JavaScript, Java, etj.
- Bazohet në:
  - objekte: `{ "key": value, ... }`
  - array: `[ value1, value2, ... ]`

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.
- Vlerat: `string`, `number`, `true`/`false`, `null`, objekte, array.

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.
- Vlerat: `string`, `number`, `true`/`false`, `null`, objekte, array.
- Nuk lejohen:

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.
- Vlerat: `string`, `number`, `true`/`false`, `null`, objekte, array.
- Nuk lejohen:
  - komente,

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.
- Vlerat: `string`, `number`, `true`/`false`, `null`, objekte, array.
- Nuk lejohen:
  - komente,
  - presje fundore,

Rregulla kryesore JSON

- Çelësat **duhet** të jenë `"string"` me thonjëza të dyfishta.
- Vlerat: `string`, `number`, `true`/`false`, `null`, objekte, array.
- Nuk lejohen:
  - komente,
  - presje fundore,
  - `NaN`, `Infinity` (standardi).

Shembull skedarë JSON

```
  "app": "KursePython",
  "version": "1.0.0",
  "last_updated": "2025-11-05T15:00:00Z",
  "users": [
    {
      "id": 1,
      "name": "Arta K.",
      "roles": ["student"],
      "active": true,
      "scores": [90, 85, 92]
    },
    {
      "id": 2,
      "name": "Bledi H.",
      "roles": ["instructor", "admin"],
      "active": false,
      "scores": null
    }
  ],
...
```

JSON ↔ objekte Python

```python
import json

data = {
  "name": "Arta",
  "age": 20,
  "scores": [90, 85, 92],
  "active": True
}

s = json.dumps(data, ensure_ascii=False, indent=2)
print(s)                    # -> string JSON

obj = json.loads(s)
print(obj["name"])
```

Skedarë JSON

```python
import json

with open('students.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

with open('students.json', 'r', encoding='utf-8') as f:
    students = json.load(f)
```

- dump/load punojnë direkt me skedarë.

Skedarë JSON

```python
import json

with open('students.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=2)

with open('students.json', 'r', encoding='utf-8') as f:
    students = json.load(f)
```

- dump/load punojnë direkt me skedarë.
- Përdor ensure$_a$*scii* = *Falseprshqip*/*Unicode*.

JSON dhe fjalorët e folezuar

```
students = {
  "A101": {"name": "Arta", "scores": [90, 85], "active": True},
  "B202": {"name": "Bledi","scores": [78, 92], "active": False}
}

s = json.dumps(students, ensure_ascii=False, indent=2)
print(s)

back = json.loads(s)
print(back["A101"]["scores"][0])
```

Gabime JSON

```
bad = '{"a": 1,}'    # presje fundore e palejuar
import json

try:
    json.loads(bad)
except json.JSONDecodeError as e:
    print("JSON i pavlefshem:", e)
```

JSON vs `pickle`

	pickle	**json**
Formati	binar	tekst
Lexueshmëria	jo	po
Portabiliteti	vetëm Python	shumëgjuhësh
Siguria	rrezik me input të jashtëm	më i sigurt
Tipet	objekte komplekse	tipe bazë

- **JSON** për API, konfigurime, formate publike.

JSON vs pickle

	pickle	**json**
Formati	binar	tekst
Lexueshmëria	jo	po
Portabiliteti	vetëm Python	shumëgjuhësh
Siguria	rrezik me input të jashtëm	më i sigurt
Tipet	objekte komplekse	tipe bazë

- **JSON** për API, konfigurime, formate publike.
- **pickle** për ruajtje lokale mes programesh Python.

1 Fjalorët (Dictionaries)

2 Set

3 Pickle: Serializimi i objekteve

4 JSON në Python

5 Përmbledhje

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.
- Dictionary comprehensions për ndërtim elegant strukturash.

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.
- Dictionary comprehensions për ndërtim elegant strukturash.
- Set-et: vlera unike, operacione matematikore, filtrime të shpejta.

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.
- Dictionary comprehensions për ndërtim elegant strukturash.
- Set-et: vlera unike, operacione matematikore, filtrime të shpejta.
- Set comprehensions për koleksione unike nga të dhëna ekzistuese.

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.
- Dictionary comprehensions për ndërtim elegant strukturash.
- Set-et: vlera unike, operacione matematikore, filtrime të shpejta.
- Set comprehensions për koleksione unike nga të dhëna ekzistuese.
- Pickle: serializim binar për objekte Python (kujdes me sigurinë).

Përmbledhje e kapitullit

- Fjalorët: çelës–vlerë, akses i shpejtë, fleksibël, folezim.
- Metodat kryesore: `get`, `items`, `values`, `pop`, `popitem`, `clear`.
- Dictionary comprehensions për ndërtim elegant strukturash.
- Set-et: vlera unike, operacione matematikore, filtrime të shpejta.
- Set comprehensions për koleksione unike nga të dhëna ekzistuese.
- Pickle: serializim binar për objekte Python (kujdes me sigurinë).
- JSON: format tekstual, standard për shkëmbim të dhënash.

Faleminderit për vëmendjen!

Pyetje?